

TNSKILLS PROJECT

AI FAQ ASSISTANT-API

AI AUGUMENTED BACKEND DEVELOPMENT WITH NODE.JS

1. INTRODUCTION

Project Title : AI FAQ ASSISTANT API

Team ID : SWTID-2026-9773

Team Leader : Saranya S - Demo video , document

Mail id: saranyasi2007@gmail.com

Team Members:

1. **Ramya P** – ramyasaanthi@gmail.com – Code Developer
2. **Rishika A** – rishikabanu7@gmail.com – code developer 2
3. **Samyuktha A** – samyukthaa34@gmail.com – Document

2. Project Overview

Purpose

The purpose of the AI FAQ Assistant is to provide a centralized backend system for managing frequently asked questions and customer-support content efficiently. It allows authorized users to create and manage FAQ records, search FAQ content, and generate FAQ content with AI.

Features

- User Registration
- User Login Authentication
- JWT-based protected API routes
- Create, Read, Update, and Delete FAQ records
- FAQ keyword/semantic search
- AI-powered FAQ generation using Google Gemini
- Category-based FAQ organization
- MongoDB database integration using Mongoose
- Password hashing using bcrypt
- Centralized error handling and schema validation

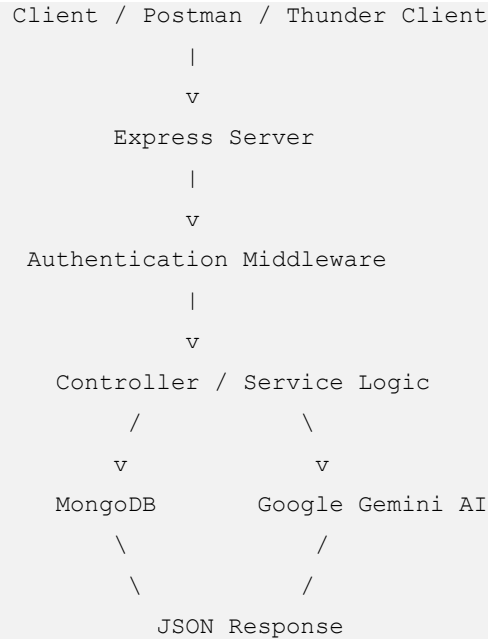
3. Backend Architecture

The backend is built using a layered REST API architecture.

Technologies Used

- Node.js – Server-side runtime environment
- Express.js – Web framework for REST API development
- MongoDB – NoSQL database for storing users and FAQ documents
- Mongoose – MongoDB object modeling, schema definition, and validation
- JWT (JSON Web Token) – Authentication and route protection
- bcrypt – Password hashing
- dotenv – Environment variable management
- cors – Cross-Origin Resource Sharing configuration
- Google Gemini SDK – AI-assisted FAQ generation

Architecture Flow



User Roles

- Admin – Can manage users, FAQs, categories, and AI-generated content.
- Content Creator – Can create, edit, delete owned FAQs, generate FAQs using AI, and organize content.
- Authenticated User – Can view/search FAQs, generate AI-assisted answers, and access personal profile information.
- Public User – Can view and search published FAQs but cannot access protected operations.

4. Setup Instructions

Prerequisites

- Node.js v18 or above
- npm v9 or above
- MongoDB
- Visual Studio Code
- Postman or Thunder Client

Installation Steps

1. Create a new project folder and open it in Visual Studio Code.
2. Create the src folder and required subfolders for middleware, models, and routes.
3. Initialize the Node.js project.

```
npm install
```

4. Install the required packages.

```
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

5. Create the .env file and configure the required environment variables such as the MongoDB connection, JWT secret, server port, and AI service configuration.
6. Start the development server using the configured npm command.

```
npm run dev
```

5. Folder Structure

The backend project is organized as follows:

```
AI-FAQ-Assistant
|
├── src
|   ├── middleware
|   |   ├── authMiddleware.js
|   |   └── errorMiddleware.js
|   |
|   ├── models
|   |   ├── User.js
|   |   └── FAQ.js
|   |
|   ├── routes
|   |   ├── authRoutes.js
|   |   ├── faqRoutes.js
|   |   └── aiRoutes.js
|   |
|   ├── app.js
|   └── server.js
|
├── .env
├── package.json
└── package-lock.json
```

Explanation

Models – Defines MongoDB/Mongoose schemas for users and FAQs.

Routes – Contains API endpoints for authentication, FAQ operations, and AI operations.

Middleware – Handles JWT authentication and centralized error processing.

app.js – Configures Express middleware and API route mounting.

server.js – Loads configuration, connects to the database, and starts the server.

.env – Stores environment-specific configuration and secrets.

6. Running the Backend Application

To start the backend development server, run:

```
npm run
```

If the server starts successfully, the terminal should show the configured server startup information and database connection status. The exact port depends on the environment configuration.

7. API Endpoints

Authentication APIs

Register User

POST /api/auth/register

Access: Public

Request Body:

```
{
  "name": "saranya",
  "email": "saranyasi@gmail.com",
  "password": "saranya2007"
}
```

Login User

POST /api/auth/login

Access: Public

Request Body:

```
{
  "email": "saranyasi@gmail.com",
  "password": "saranya2007"
}
```

A successful login returns a JWT token for protected API requests.

8. FAQ and AI APIs

Create FAQ

POST /api/faqs

Access: Private – Bearer JWT Token

Request Body:

```
{
  "question": "what is mongodb?",
  "answer": "\"MongoDB is a popular NoSQL document-oriented database that stores data In flexible, 350N-like documents rather than traditional tables and rows. It is designed for high performance, high availability, and easy scalability\"",
  "category": "Configuration"
}
```

Search FAQs

GET /api/faqs/search?q=configure

Access: Public

This endpoint retrieves FAQ documents matching the search query.

Generate FAQ Using AI

POST /api/ai/generate-faq

Access: Private – Bearer JWT Token

Request Body:

```
{
  "topic": "Express.js"
}
```

The AI service generates FAQ content based on the supplied topic.

9. Authentication

The backend uses JWT Authentication to protect private API endpoints.

Process

7. The user registers with a name, email, and password.
8. The user logs in using the registered email and password.
9. The server validates the credentials and generates a JWT token.
10. The client sends the token in the request Authorization header for protected routes.
11. The authentication middleware verifies the token before allowing access.

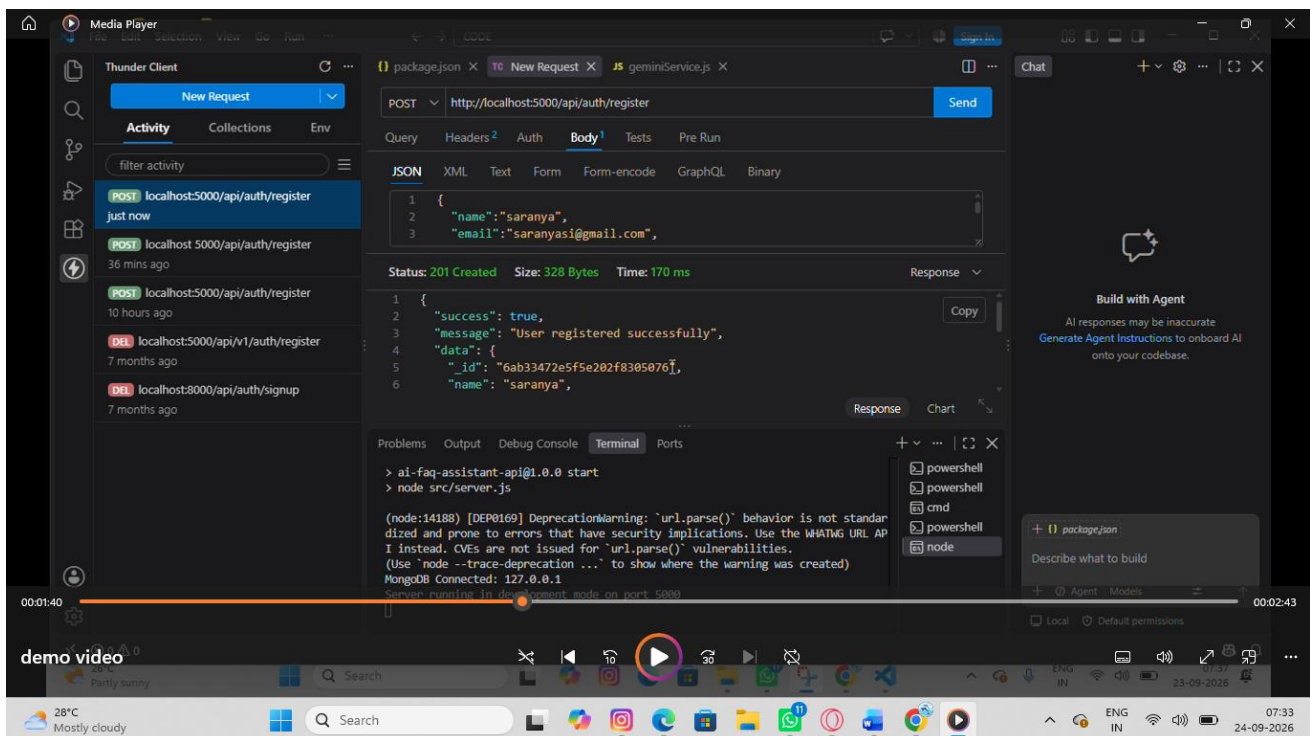
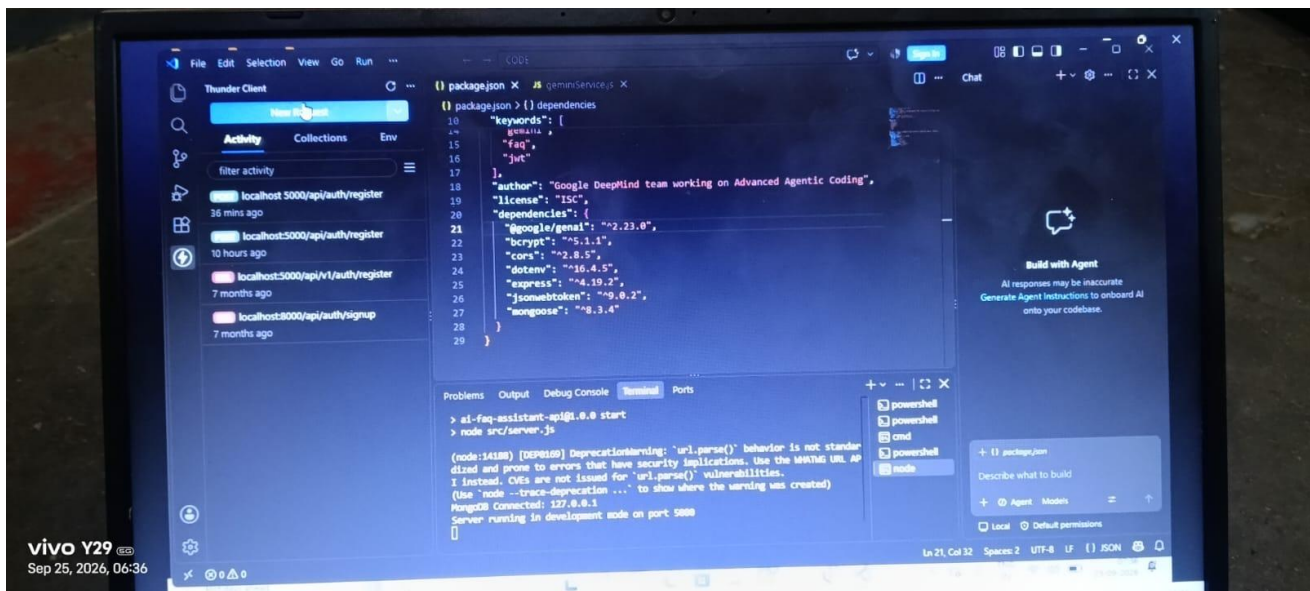
Example

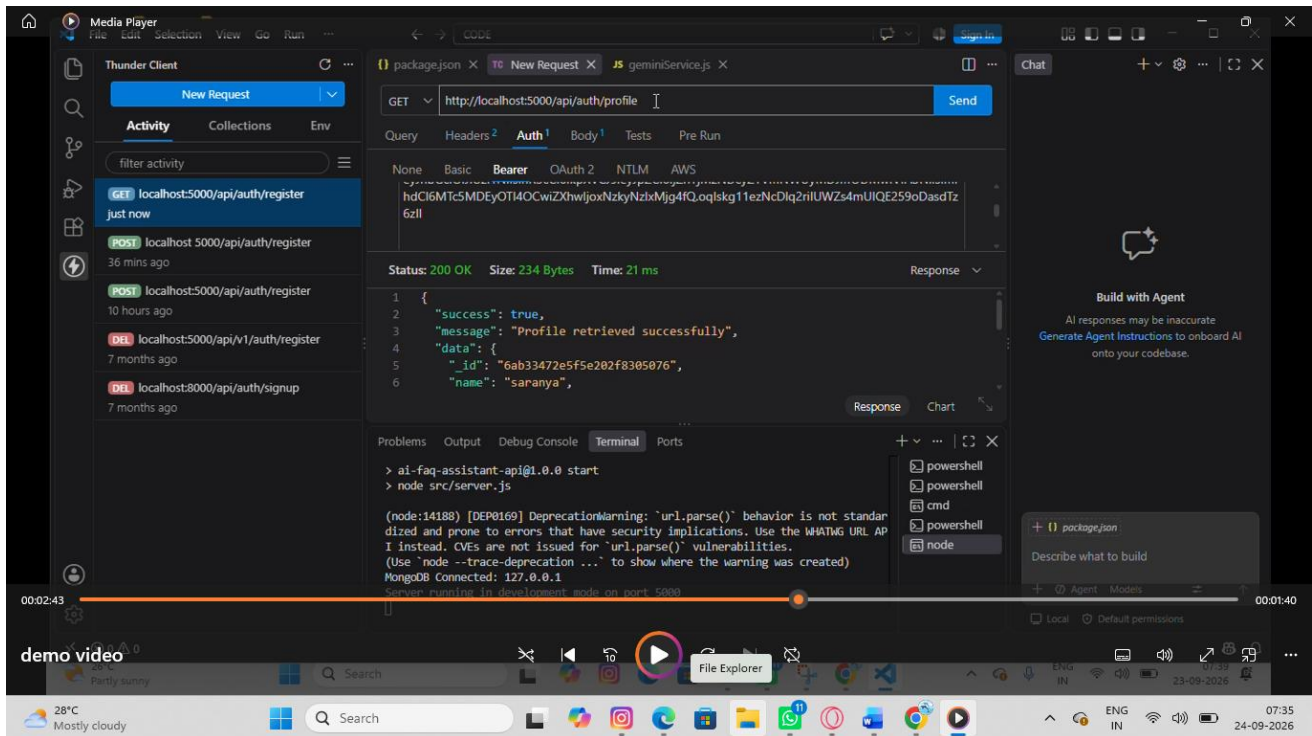
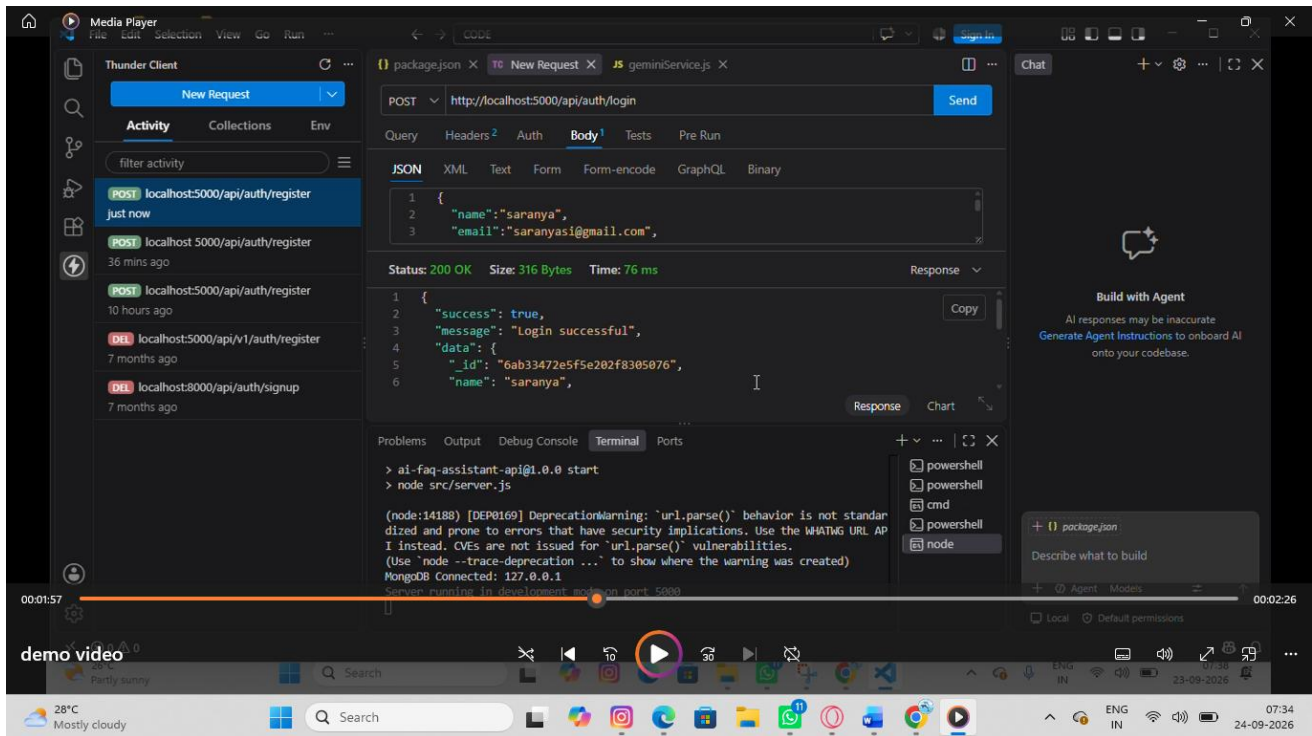
```
Authorization: Bearer TOKEN
```

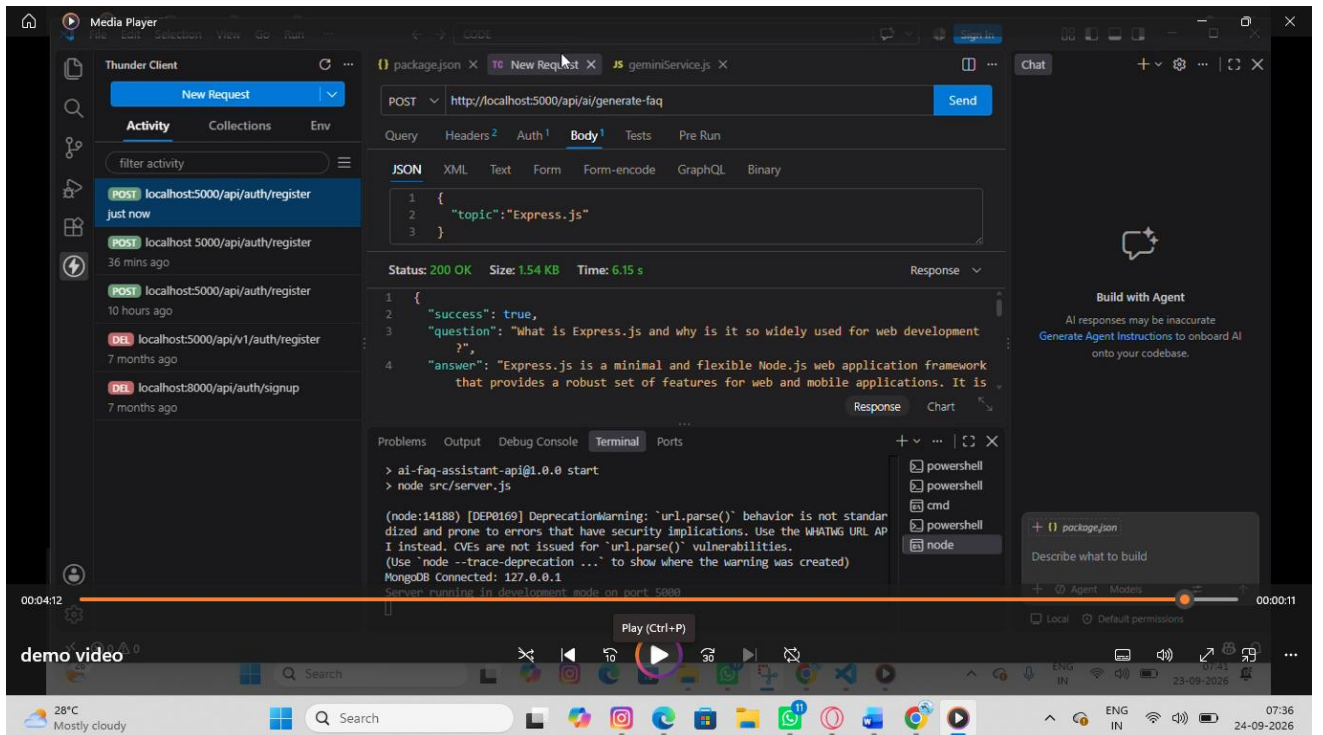
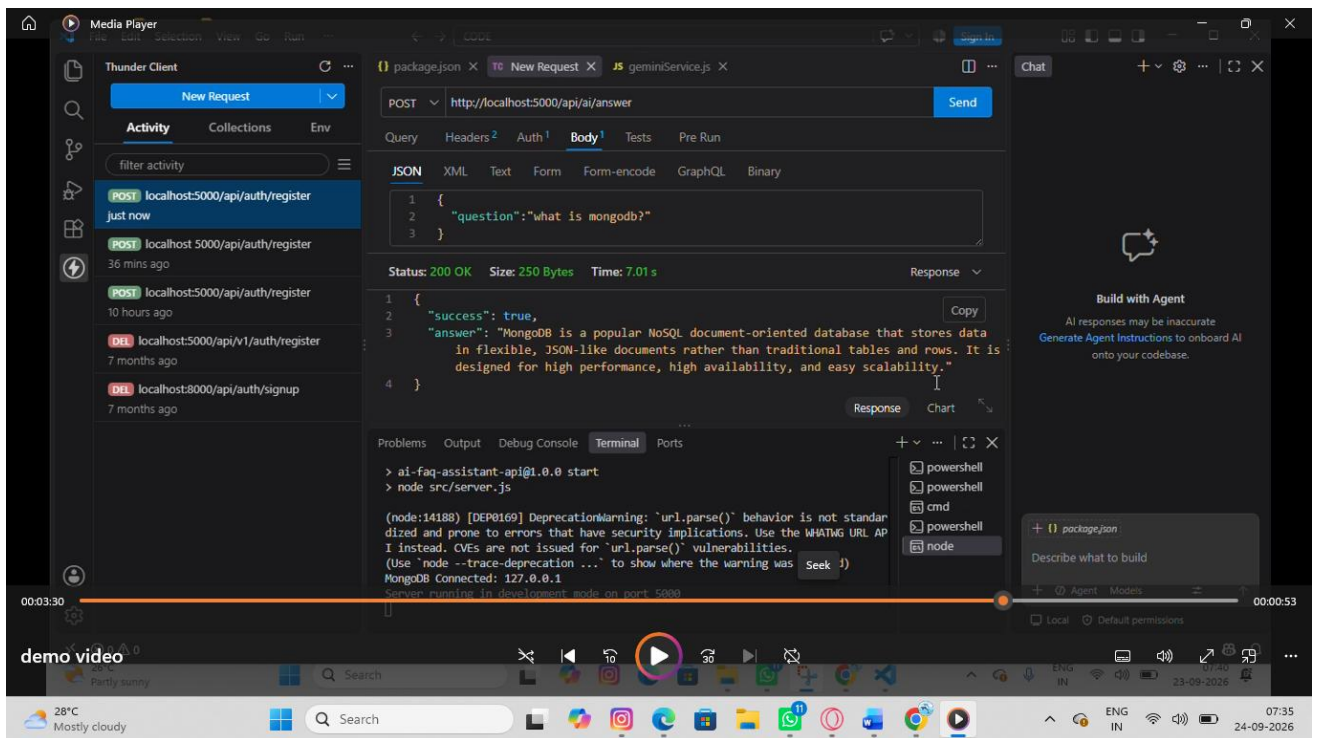
10. SCREENSHOTS

Add screenshots of the working project here. The recommended screenshots for the final documentation are:

- User registration request and successful response.
- User login request showing JWT authentication response.
- Create FAQ request and response.
- FAQ search request and returned results.
- AI FAQ generation request and generated response.
- MongoDB collection showing stored user/FAQ records, where appropriate.
- Project running successfully in the terminal.







11. Database Design

The backend uses MongoDB for data storage with Mongoose ODM.

User Collection

Fields:

- `_id` (ObjectId)
- `name`
- `email`
- `password` (hashed)
- `role`
- `createdAt`
- `updatedAt`

FAQ Collection

Fields:

- `_id` (ObjectId)
- `question`
- `answer`
- `category`
- `createdBy` (reference to User)
- `createdAt`
- `updatedAt`

AI Generation (Logical Entity)

AI-generated content is produced dynamically before it is optionally persisted as FAQ content.

- `topic`
- `generatedQuestion`
- `generatedAnswer`
- `generatedCategory`
- `generatedAt`

Relationships

- User → FAQ: One user can create multiple FAQ records.
- Category → FAQ: One category can contain multiple FAQ records.
- User → AI Generated FAQ: One user can generate multiple AI-assisted FAQs.
- FAQ → Search Results: Search queries retrieve matching FAQ documents.

12. Security Features

Security techniques implemented:

- Password hashing using bcrypt
- JWT authentication
- Protected API routes
- Authorization checks for protected operations
- Environment variable protection using dotenv
- Mongoose schema validation
- Centralized error handling to reduce exposure of internal implementation details

13. Testing

API testing is performed using Thunder Client / Postman.

Tested Endpoints

- User Registration – POST /api/auth/register
- User Login – POST /api/auth/login
- Create FAQ – POST /api/faqs
- FAQ Search – GET /api/faqs/search?q=configure
- AI FAQ Generation – POST /api/ai/generate-faq

Testing Areas

- Authentication and JWT token validation
- Protected route access
- FAQ creation and validation
- FAQ search
- AI generation
- Invalid input and error responses
- Database persistence

The reference project describes the API endpoints and their expected request structures for testing.

14. Known Issues

- AI FAQ generation depends on the availability and correct configuration of the external Google Gemini AI service.
- MongoDB must be available and correctly configured for persistent data storage.
- Environment variables must be configured correctly before running the application.

- The reference project is a headless backend service, so a separate frontend is not included in the backend implementation.
- Search functionality is described as keyword/semantic search; the exact advanced semantic-search implementation is not specified in the reference document.

15. Future Enhancements

- Develop a dedicated web frontend for the REST API.
- Add advanced semantic search using embeddings and vector search.
- Add an analytics dashboard for FAQ usage and search trends.
- Expand role and permission management.
- Add AI-based summarization and automatic FAQ categorization.
- Add notifications and automated content-review workflows.
- Deploy the backend and database to a cloud environment with secure configuration.